

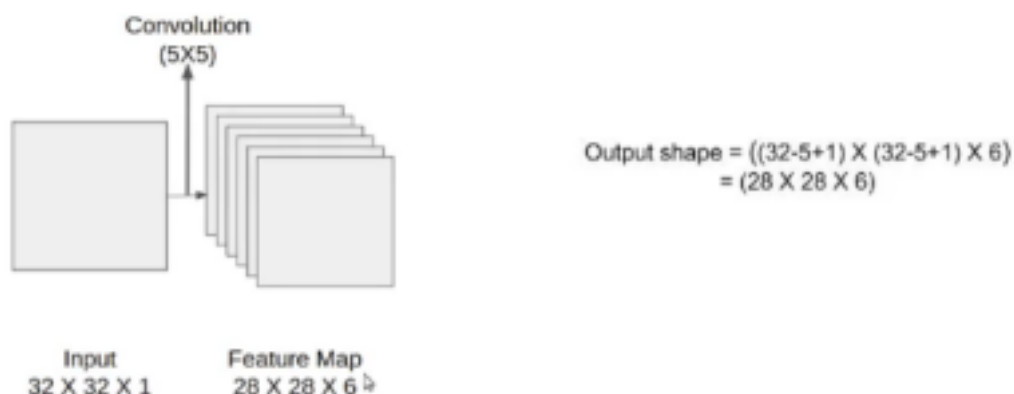
LeNet-5

LeNet-5, a pioneering 7-level convolutional network by LeCun et al in 1998, that classifies digits, was applied by several banks to recognize hand-written numbers on checks (cheques) digitized in 32x32 pixel greyscale input images. The ability to process higher-resolution images requires larger and more convolutional layers, so this technique is constrained by the availability of computing resources.

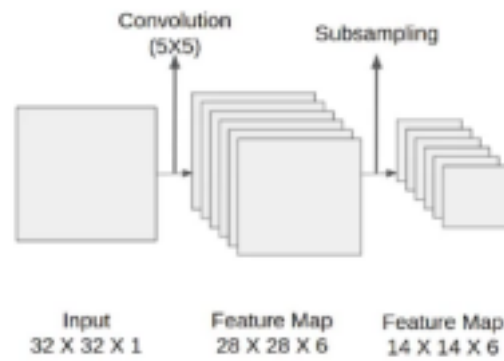
The network has 5 layers with learnable parameters and is hence named Lenet-5. It has three sets of convolution layers with a combination of average pooling. After the convolution and average pooling layers, we have two fully connected layers. At last, a Softmax classifier which classifies the images into respective class.



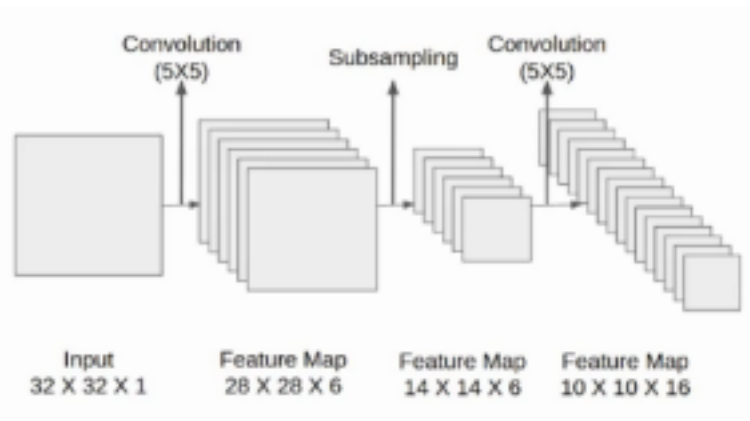
The input to this model is a 32 X 32 grayscale image hence the number of channels is one.



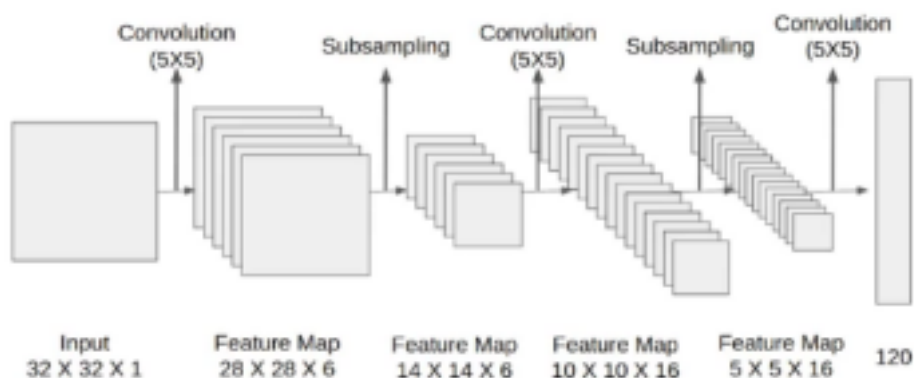
We then apply the first convolution operation with the filter size 5X5 and we have 6 such filters. As a result, we get a feature map of size 28X28X6. Here the number of channels is equal to the number of filters applied.



After the first pooling operation, we apply the average pooling and the size of the feature map is reduced by half. Note that, the number of channels is intact.



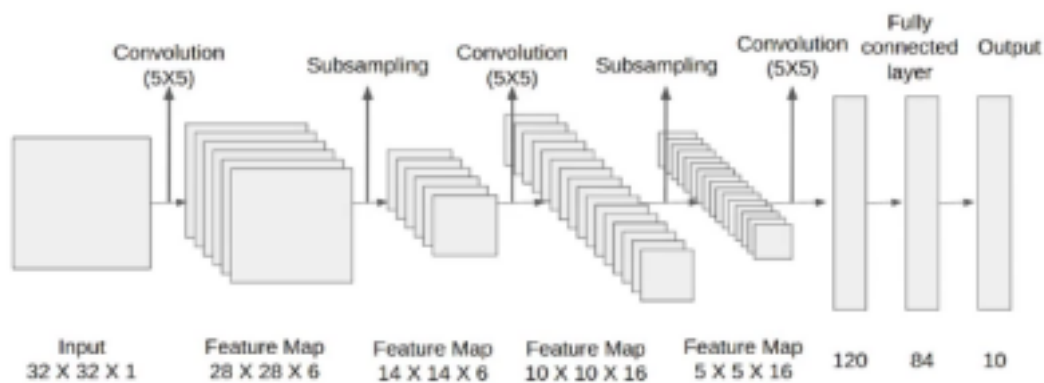
Next, we have a convolution layer with sixteen filters of size 5X5. Again the feature map changed it is 10X10X16. The output size is calculated in a similar manner. After this, we again applied an average pooling or subsampling layer, which again reduce the size of the feature map by half i.e 5X5X16.



Then we have a final convolution layer of size 5X5 with 120 filters. As shown in the above image. Leaving the feature map size 1X1X120. After which flatten result is 120 values.

After these convolution layers, we have a fully connected layer with eighty-four neurons. At last, we have an output layer with ten neurons since the data have ten classes.

Here is the final architecture of the Lenet-5 model.



AlexNet

The AlexNet has eight layers with learnable parameters. The model consists of five layers with a combination of max pooling followed by 3 fully connected layers and they use Relu activation in each of these layers except the output layer.

They found out that using the relu as an activation function accelerated the speed of the training process by almost six times. They also used the dropout layers, that prevented their model from overfitting. Further, the model is trained on the Imagenet dataset. The Imagenet dataset has almost 14 million images across a thousand classes.

The input to this model is the images of size 227X227X3. Then we apply the first convolution layer with 96 filters of size 11X11 with stride 4. The activation function used in this layer is relu. The output feature map is 55X55X96.

In case, you are unaware of how to calculate the output size of a convolution

$$\text{layer output} = ((\text{Input-filter size}) / \text{stride}) + 1$$

Also, the number of filters becomes the channel in the output feature map.

Next, we have the first Maxpooling layer, of size 3X3 and stride 2. Then we get the resulting feature map with the size 27X27X96.

After this, we apply the second convolution operation. This time the filter size is reduced to 5X5 and we have 256 such filters. The stride is 1 and padding 2. The activation function used is again relu. Now the output size we get is 27X27X256.

Again we applied a max-pooling layer of size 3X3 with stride 2. The resulting feature map is of shape 13X13X256.

Now we apply the third convolution operation with 384 filters of size 3X3 stride 1 and also padding 1. Again the activation function used is relu. The output feature map is of shape 13X13X384. Then we have the fourth convolution operation with 384 filters of size 3X3. The stride along with the padding is 1. On

top of that activation function used is relu. Now the output size remains unchanged i.e 13X13X384.

After this, we have the final convolution layer of size 3X3 with 256 such filters. The stride and padding are set to one also the activation function is relu. The resulting feature map is of shape 13X13X256.

So if you look at the architecture till now, the number of filters is increasing as we are going deeper. Hence it is extracting more features as we move deeper into the architecture. Also, the filter size is reducing, which means the initial filter was larger and as we go ahead the filter size is decreasing, resulting in a decrease in the feature map shape.

Next, we apply the third max-pooling layer of size 3X3 and stride 2. Resulting in the feature map of the shape 6X6X256.

After this, we have our first dropout layer. The drop-out rate is set to be 0.5.

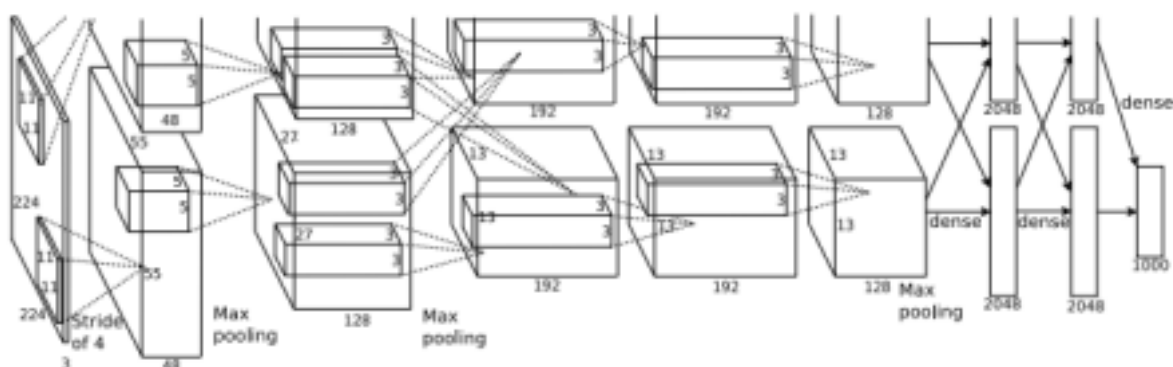
Then we have the first fully connected layer with a relu activation function. The size of the output is 4096. Next comes another dropout layer with the dropout rate fixed at 0.5.

This is followed by a second fully connected layer with 4096 neurons and relu activation.

Finally, we have the last fully connected layer or output layer with 1000 neurons as we have 10000 classes in the data set. The activation function used at this layer is Softmax.

This is the architecture of the Alexnet model. It has a total of 62.3 million learnable

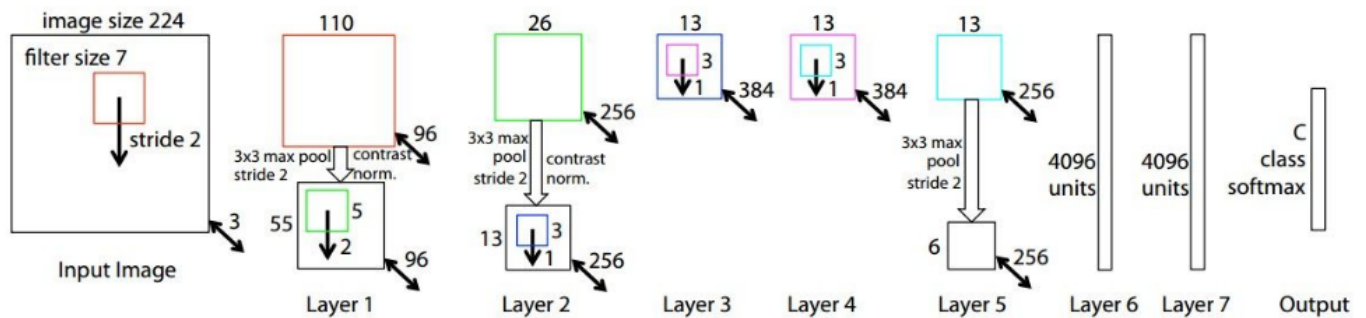
parameters.



ZFNet

ZFNet entered the ImageNet competition in 2013, the next year after AlexNet had won the competition. A professor at NYU named Dr. Rob Fergus along with his Ph.D. student Dr. Matthew D. Zeiler designed this new deep neural network and named it after the initials of their surnames.

It surpassed the results of AlexNet with an 11.2% error rate and was the winner of the 2013 ImageNet challenge.



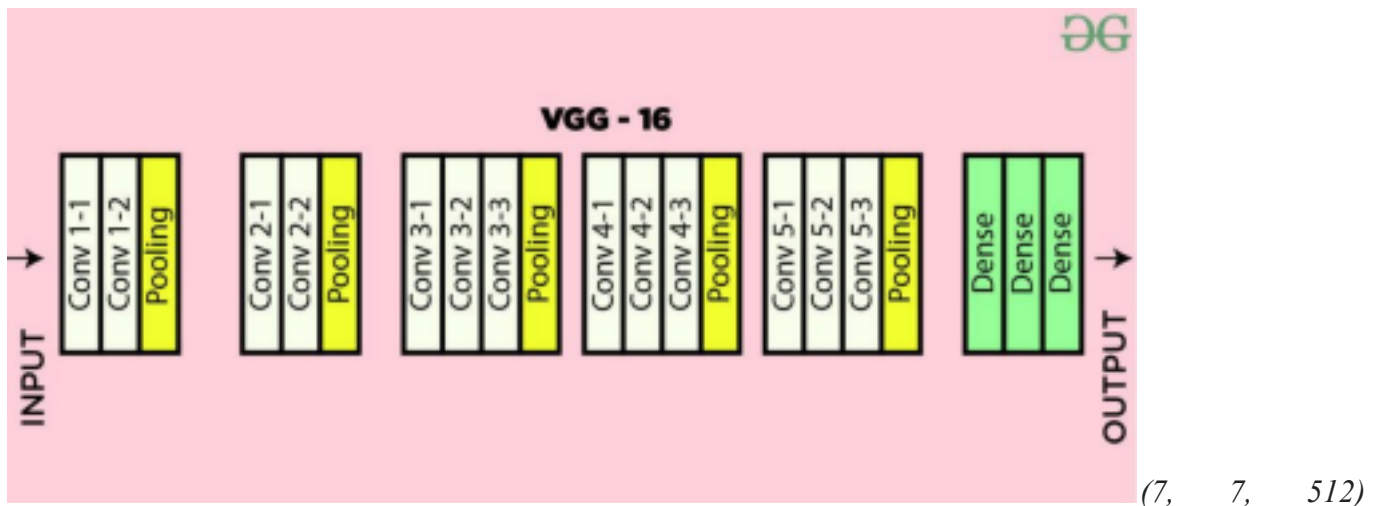
ZFNet is considered as an extended version of AlexNet with some modifications to filter size to achieve better accuracy. ZFNet used 7×7 sized filters, on the other hand, AlexNet used 11×11 filters. The idea for using smaller filters in the convolutional layer was to avoid the loss of pixel information.

Although ZFNet was able to improve the way of extracting pixel information, it couldn't decrease the computational cost that was involved in going deeper into the network.

VGG-16

The input to the network is an image of dimensions $(224, 224, 3)$. The first two layers have 64 channels of 3×3 filter size and the same padding. Then after a max pool layer of stride $(2, 2)$, two layers have convolution layers of 256 filter size and filter size $(3, 3)$. This is followed by a max-pooling layer of stride $(2, 2)$ which is the same as the previous layer. Then there are 2 convolution layers of filter size $(3, 3)$ and 256 filters. After that, there are 2 sets of 3 convolution layers and a max pool layer. Each has 512 filters of $(3, 3)$ size with the same padding. This image is then passed to the stack of two convolution layers. In these convolution and max-pooling layers, the filters we use is of the size 3×3 instead of 11×11 in AlexNet and 7×7 in ZF-Net. In some of the layers, it also uses 1×1 pixel which is used to manipulate the number of input channels. There is a padding of 1-pixel (same padding) done after each convolution layer to prevent the spatial feature of the image.

After the stack of convolution and max-pooling layer, we got a



feature map. We flatten this output to make it a $(1, 25088)$ feature vector. After this there are 3 *fully* connected layers, the first layer takes input from the last feature vector and outputs a $(1, 4096)$ vector, the second layer also outputs a vector of size $(1, 4096)$ but the third layer output 1000 channels for 1000 classes of ILSVRC challenge, then after the output of 3rd fully connected layer is passed to softmax layer in order to normalize the classification vector. After the output of classification vector top-5 categories for evaluation. All the hidden layers use ReLU as its activation function. ReLU is more computationally efficient because it results in faster learning and it also decreases the likelihood of vanishing gradient problems.

GoogleNet

Google Net (or Inception V1) was proposed by research at Google (with the collaboration of various universities) in 2014 in the research paper titled “Going Deeper with Convolutions”. This architecture was the winner at the ILSVRC 2014 image classification challenge. It has provided a significant decrease in error rate as compared to previous winners AlexNet (Winner of ILSVRC 2012) and ZF-Net (Winner of ILSVRC 2013) and significantly less error rate than VGG (2014 runner up). This architecture uses techniques such as 1×1 convolutions in the middle of the architecture and global average pooling.

Model Architecture:

The overall architecture is 22 layers deep. The architecture was designed to keep computational efficiency in mind. The idea behind that the architecture can be run on individual devices even with low computational resources. The architecture also contains two auxiliary classifier layer connected to the output of Inception (4a) and Inception (4d) layers.

The architectural details of auxiliary classifiers as follows:

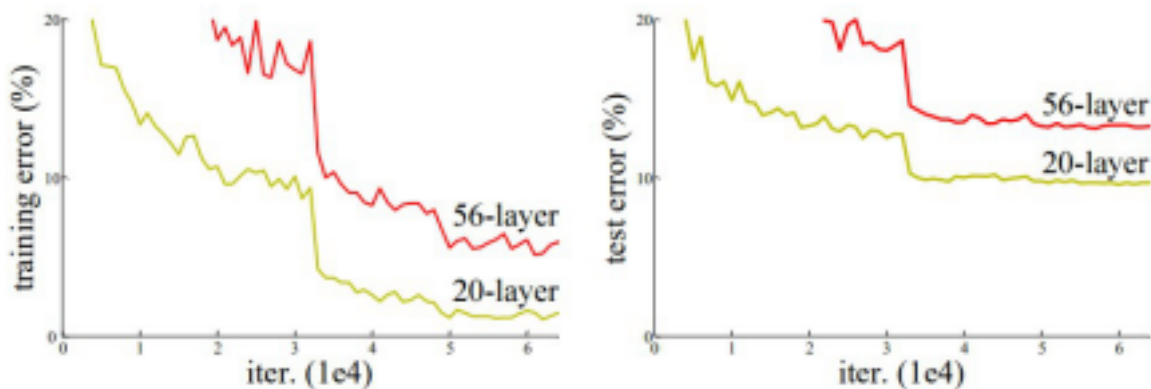
- An average pooling layer of filter size 5×5 and stride 3.
- A 1×1 convolution with 128 filters for dimension reduction and ReLU activation.
- A fully connected layer with 1025 outputs and ReLU activation
- Dropout Regularization with dropout ratio = 0.7
- A softmax classifier with 1000 classes output similar to the main softmax classifier.



This architecture takes images of size 224×224 with RGB color channels. All the convolutions inside this architecture use Rectified Linear Units (ReLU) as their activation functions.

ResNet

After the first CNN-based architecture (AlexNet) that won the ImageNet 2012 competition, Every subsequent winning architecture uses more layers in a deep neural network to reduce the error rate. This works for fewer layers, but when we increase the number of layers, there is a common problem in deep learning associated with that called Vanishing/Exploding gradient. This causes the gradient to become 0 or too large. Thus when we increase the number of layers, the training and test error rate also increases.



In the above plot, we can observe that a 56-layer CNN gives more error rate on both training and testing datasets than a 20-layer CNN architecture. If this was the result of overfitting, then we should have a lower training error in 56-layer CNN but then it also has higher training error. After analyzing more on the error rate the authors were able to reach the conclusion that it is caused by vanishing/exploding gradient. ResNet, which was proposed in 2015 by researchers at Microsoft Research, introduced a new architecture called Residual Network.

This network uses a 34-layer plain network architecture inspired by VGG-19 in which then the shortcut connection is added. These shortcut connections then convert the architecture into the residual network.

Visualizing Convolutional Neural Networks

Typically, the reasons listed below are the most important points for a deep learning practitioner to remember:

1. Understanding how the model works
2. Assistance in Hyperparameter tuning
3. Finding out the failures of the model and getting an intuition of why they fail
4. Explaining the decisions to a consumer / end-user or a business executive

Let us look at an example where visualizing a neural network model helped in understanding the follies and improving the performance

Once upon a time, the US Army wanted to use neural networks to automatically detect camouflaged enemy tanks. The researchers trained a neural net on 50 photos of camouflaged tanks in trees, and 50 photos of trees without tanks. Using standard techniques for supervised learning, the researchers trained the neural network to a weighting that correctly loaded the training set—output “yes” for the 50 photos of camouflaged tanks, and output “no” for the 50 photos of forest.

This did not ensure, or even imply, that *new* examples would be classified correctly. The neural network might have “learned” 100 special cases that would not generalize to any new problem. Wisely, the researchers had originally taken 200 photos, 100 photos of tanks and 100 photos of trees. They had used only 50 of each for the training set. The researchers ran the neural network on the remaining 100 photos, and without further training the neural network classified all remaining photos correctly. Success confirmed! The researchers handed the finished work to the Pentagon, which soon handed it back, complaining that in their own tests the neural network did no better than chance at discriminating photos.



It turned out that in the researchers' dataset, photos of camouflaged tanks had been taken on cloudy days, while photos of plain forest had been taken on sunny days. The neural network had learned to distinguish cloudy days from sunny days, instead of distinguishing camouflaged tanks from an empty forest.

Methods of Visualizing a CNN model

Broadly the methods of Visualizing a CNN model can be categorized into three parts based on their internal workings

- Preliminary methods – Simple methods which show us the overall structure of a trained model
- Activation based methods – In these methods, we decipher the activations of the individual neurons or a group of neurons to get an intuition of what they are doing
- Gradient based methods – These methods tend to manipulate the gradients that are formed from a forward and backward pass while training a model

1. Preliminary Methods

1.1 Plotting model architecture

The simplest thing you can do is to print/plot the model. Here, you can also print the shapes of individual layers of the neural network and the parameters in each layer.

1.2 Visualize filters

Another way is to plot the filters of a trained model, so that we can understand the behaviour of those filters.

2. Activation Maps

2.1 Maximal Activations

To see what our neural network is doing, we can apply the filters over an input image and then plot the output. This allows us to understand what sort of input patterns activate a particular filter. For example, there could be a face filter that activates when it gets the presence of a face in the image.

2.2 Image Occlusion

In an image classification problem, a natural question is if the model is truly identifying the location of the object in the image, or just using the surrounding context. We took a brief look at this in gradient based methods above. Occlusion based methods attempt to answer this question by systematically occluding different portions of the input image with a grey square, and monitoring the output of the classifier. The

examples clearly show the model is localizing the objects within the scene, as the probability of the correct class drops significantly when the object is occluded.

3. Gradient Based Methods

3.1 Saliency Maps

The concept of using saliency maps is pretty straight-forward – we compute the gradient of the output category with respect to the input image. This should tell us how the output category value changes with respect to a small change in the input image pixels. All the positive values in the gradients tell us that a small change to that pixel will increase the output value. Hence, visualizing these gradients, which are the same shape as the image, should provide some intuition of attention.

3.2 Gradient based Class Activations Maps

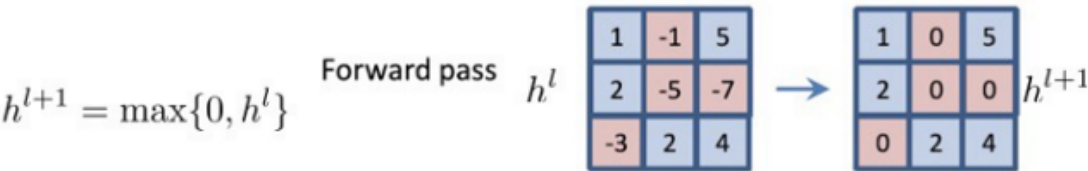
Class activation maps, or grad-CAM, is another way of visualizing what our model looks at while making predictions. Instead of using gradients with respect to the output, grad-CAM uses penultimate Convolutional layer output. This is done to utilize the spatial information that is being stored in the penultimate layer.

Guided Backpropagation:

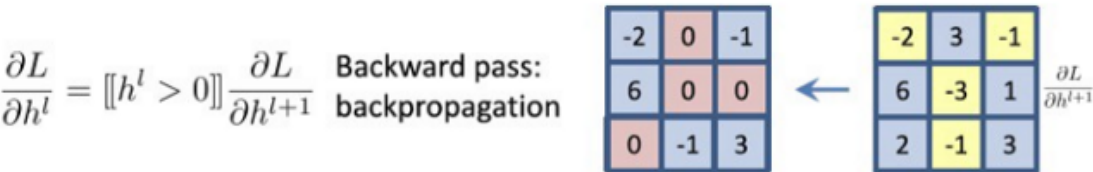
Guided Backpropagation combines vanilla backpropagation at ReLUs (leveraging which elements are positive in the preceding feature map) with DeconvNets (keeping only positive error signals). We are only interested in what image features the neuron detects. So when propagating the gradient, we set all the negative gradients to 0. We don't care if a pixel "suppresses" (negative value) a neuron somewhere along the part to our neuron. Value in the filter map greater than zero signifies the pixel importance which is overlapped with the input image to show which pixel from the input image contributed the most.

Given below is the example of how guided backpropagation works:

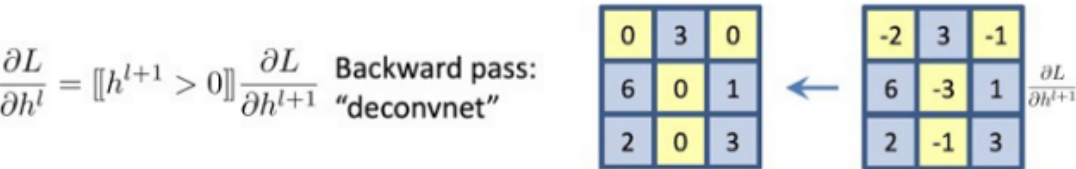
Relu Forward Pass (Flow the value which are greater than 0):



Relu Backward Pass (flow the value as it is where value is greater than zero in the filter (h_l) during forward propagation.)



Deconvolution for Relu: Flow the values backward as it is where value in the filter is greater than 0.



Guided Backpropagation: Taking the intersection of the concept of Backward pass and the deconvolution.

$$\frac{\partial L}{\partial h^l} = \mathbb{I}[(h^l > 0) \&\& (h^{l+1} > 0)]$$

Backward pass:
guided
backpropagation

$$\frac{\partial L}{\partial h^{l+1}}$$

0	0	0
6	0	0
0	0	3

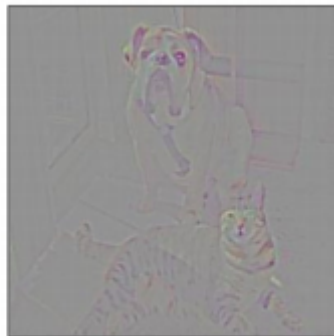
$$\leftarrow$$

-2	3	-1
6	-3	1
2	-1	3

$$\frac{\partial L}{\partial h^{l+1}}$$



(a) Original Image



(b) Guided Backprop 'Cat'

DeepDream

DeepDream is an experiment that visualizes the patterns learned by a neural network. Similar to when a child watches clouds and tries to interpret random shapes, DeepDream over-interprets and enhances the patterns it sees in an image.

It does so by forwarding an image through the network, then calculating the gradient of the image with respect to the activations of a particular layer. The image is then modified to increase these activations, enhancing the patterns seen by the network, and resulting in a dream-like image. This process was dubbed "Inceptionism" (a reference to InceptionNet, and the movie Inception).

The idea in DeepDream is to choose a layer (or layers) and maximize the "loss" in a way that the image increasingly "excites" the layers. The complexity of the features incorporated depends on layers chosen by you, i.e, lower layers produce strokes or simple patterns, while deeper layers give sophisticated features in images, or even whole objects.

DeepArt

DeepArt is a website that allows users to create artistic images by using an algorithm to redraw one image using the stylistic elements of another image. This uses "A Neural Algorithm of Artistic Style" a Neural Style Transfer algorithm that was developed by several of its creators to separate style elements from a piece of art. The tool allows users to create imitation works of art using the style of various artists. The neural algorithm is used by the Deep Art website to create a representation of an image provided by the user by using the 'style' of another image provided by the user. A similar program, Prisma, is an iOS and Android app that was based on the open source programming that underlies DeepArt.

Fooling Convolutional Neural Networks

